

HLLM: Enhancing Sequential Recommendations via Hierarchical Large Language Models for Item and User Modeling

Junyi Chen*, Lu Chi*, Bingyue Peng, Zehuan Yuan†

ByteDance

{chenjunyi.s, chilu, bingyue.peng, yuanzehuan}@bytedance.com

Abstract

Large Language Models (LLMs) have achieved remarkable success in various fields, prompting several studies to explore their potential in recommendation systems. However, these attempts have so far resulted in only modest improvements over traditional recommendation models. Moreover, three critical questions remain under-explored: firstly, the real value of LLMs’ pre-trained weights, often considered to encapsulate world knowledge; secondly, the necessity of fine-tuning for recommendation tasks; lastly, whether LLMs can exhibit the same scalability benefits in recommendation systems as they do in other domains. In this paper, we propose a novel **Hierarchical Large Language Model (HLLM)** architecture designed to enhance sequential recommendation systems. Our approach employs a two-tier model: the first Item LLM extracts rich content features from the detailed text description of the item, while the second User LLM utilizes these features to predict users’ future interests based on their interaction history. Extensive experiments demonstrate that our method effectively leverages the pre-trained capabilities of open-source LLMs, and further fine-tuning leads to significant performance boosts. Additionally, HLLM achieves excellent scalability, with the largest configuration utilizing 7B parameters for both item feature extraction and user interest modeling. Moreover, HLLM offers excellent training and serving efficiency, making it practical in real-world applications. Evaluations on two large-scale datasets, PixelRec and Amazon Reviews, show that HLLM achieves state-of-the-art results, outperforming traditional ID-based models by a wide margin. In online A/B testing, HLLM showcases notable gains, validating its practical impact in real-world recommendation scenarios. Codes are available at <https://github.com/bytedance/HLLM>.

1 Introduction

The recommendation algorithm is a classic yet complex problem that requires understanding user interests to predict future behaviors across various items. The key to effective recommendation lies in accurately modeling both item and user features. Currently, mainstream approaches are predominantly ID-based, converting items and users into IDs and creating corresponding embedding tables for encoding (Goldberg et al. 1992; Koren, Bell, and Volinsky 2009;

Sarwar et al. 2001). To capture diverse and temporally varying user interests, several sequential modeling methods have been developed, demonstrating notable success in sequential recommendations (Hidasi et al. 2015; Zhou et al. 2018; Kang and McAuley 2018; Sun et al. 2019). However, these methods are typically dominated by embedding parameters and have relatively small model sizes, leading to two major drawbacks: a heavy reliance on ID features which results in poor performance in cold-start scenarios, and relatively shallow neural networks find it difficult to model complex and diverse user interests.

With the advent of ChatGPT (OpenAI 2022), large language models (LLMs) have achieved significant breakthroughs across various domains, showcasing impressive world knowledge and reasoning capabilities (Touvron et al. 2023; Achiam et al. 2023; Team et al. 2023). This success has spurred interest among researchers in exploring the integration of LLMs into recommendation systems (Wu et al. 2023; Li et al. 2023b). These explorations can be broadly categorized into **three approaches**: (1). Utilizing LLMs to provide refined or supplementary information for recommendation systems (Zhang et al. 2024a; Ren et al. 2024; Xi et al. 2023), such as summary of user behavior and item information expansion. (2). Transforming the recommendation system into a dialogue-driven format compatible with LLMs (Bao et al. 2023; Friedman et al. 2023; Zhang et al. 2023; Yang et al. 2023; Zhai et al. 2023). (3). Modifying LLMs to handle recommendation tasks beyond just text input and output. This includes approaches that input ID features into LLMs (Ning et al. 2024; Zhai et al. 2024; Liao et al. 2024) and those that replace existing models with LLMs, optimizing directly for objectives like Click-Through Rate (CTR) (Cui et al. 2022; Kang et al. 2023).

Despite these advancements, integrating LLMs with recommendation systems presents notable challenges in complexity and effectiveness. One issue is that inputting user behavior history as text to LLMs results in very long input sequences. Consequently, LLMs need longer sequences to represent the same time span of user behavior than ID-based methods, while the complexity of the self-attention module in LLMs scales quadratically with the sequence length. Additionally, recommending a single item requires generating several text tokens, leading to multiple forwards and resulting in lower efficiency. In terms of effectiveness, the

*Equal contribution. †Corresponding author.

performance improvements of existing LLM-based methods over traditional methods are not significant, raising questions about whether the potential of LLMs has been fully realized.

Moreover, some critical issues remain underexplored. Firstly, the actual value of pre-trained LLM weights, often regarded as encapsulating world knowledge, needs further investigation. While LLMs offer impressive zero-shot and few-shot capabilities, their value when training on large-scale recommendation data is unclear. Secondly, the necessity of fine-tuning for recommendation tasks is in question. LLMs pre-trained on massive corpora exhibit strong world knowledge, but whether further fine-tuning on recommendation tasks enhances or diminishes performance remains to be seen. Lastly, the scalability of LLMs, a hallmark characteristic with proven scaling laws in other domains, requires validation in the context of recommendation systems. While some studies have successfully validated the scaling laws in the recommendation domain (Shin et al. 2023; Zhai et al. 2024), these models have considerably fewer parameters compared to LLMs. Whether models exceeding 1 billion parameters exhibit good scalability in the recommendation domain remains an open question.

To address these challenges, this paper proposes the **Hierarchical Large Language Model (HLLM)** architecture. The approach begins by using an LLM to extract item features. To empower the LLM to effectively extract these features, a special token is appended to the end of the detailed textual description of each item. This augmented description is then input into the LLM (referred to as the Item LLM), and the output corresponding to the special token is used as the item feature. These item features are then input into a second LLM (referred to as the User LLM) to model user interest and predict future behaviors. By transforming extensive item descriptions into concise embeddings, the length of behavior sequences is reduced to that of ID-based models, significantly lowering computational complexity compared to other text-based LLM recommendation models. We also verified that HLLM has a significant training efficiency advantage compared to ID-based models, as it can surpass ID-based models with only a small amount of training data.

Extensive experiments are conducted to explore the value of pre-training. Although the HLLM does not employ text interaction in the conventional manner of standard LLMs, such as the Item LLM being designed as a feature extractor, and both input and output of the User LLM being item embeddings, the pre-trained weights have proven beneficial for both types of LLMs. This demonstrates that the world knowledge embedded in LLMs is indeed valuable for recommendations. Nevertheless, this does not obviate the need for fine-tuning towards recommendation objectives. Conversely, our experiments indicate that such fine-tuning is crucial for surpassing traditional methods. To verify scalability, experiments on large academic datasets confirm that LLMs exhibit excellent scalability with performance improving as model parameters increase. Within the limited resources, models up to 7 billion parameters show consistent performance gains with increasing size.

Ultimately, the proposed HLLM architecture outperforms

existing methods across multiple academic datasets, achieving state-of-the-art results. More importantly, the effectiveness of HLLM is also validated through real-world online A/B testing, confirming its practical applicability.

Our main contributions can be summarized as follows:

1) A novel hierarchical LLM (HLLM) framework is introduced for sequential recommendations. This approach significantly outperforms classical ID-based models on large-scale academic datasets and has been validated to yield tangible benefits in real-world industrial settings. Additionally, this method demonstrates excellent training and serving efficiency.

2) HLLM effectively transfers the world knowledge encoded during the LLM pre-training stage into the recommendation model, encompassing both item feature extraction and user interest modeling. Nevertheless, task-specific fine-tuning with recommendation objectives is essential.

3) HLLM exhibits excellent scalability, with performance continuously improving as the data volume and model parameters increase. This scalability highlights the potential of the proposed approach when applied to even larger datasets and model sizes.

2 Related Work

Traditional Recommender Systems

Traditional Recommender Systems predominantly rely on ID-based embeddings, and how to design feature interactions is an important topic. DeepFM (Guo et al. 2017) models low-order feature interactions with FM and models high-order feature interactions with DNN. DCN (Wang et al. 2017, 2021) can model higher-order interactions by explicitly applying feature crossing at each layer. Besides, some researchers make efforts to model user interests from their historical behavior. For instance, DIN (Zhou et al. 2018) and DIEN (Zhou et al. 2019) introduce attention mechanisms to capture user’s diverse interests from historical behaviors. Inspired by transformer, SASRec (Kang and McAuley 2018) applies self-attention mechanisms to sequential recommendation. CLUE (Shin et al. 2023) and HSTU (Zhai et al. 2024) demonstrate that models with parameter counts within hundreds of millions adhere to the scaling law. Some works have also introduced content features into recommendation models, showing certain advantages in generalization (Baltescu et al. 2022; Li et al. 2023a; Cheng et al. 2024).

Recommendation with Language Models

The success of LLMs has attracted many researchers to explore their applications in recommendation systems. These explorations can be categorized into three types. Firstly, LLMs are used for summarizing or supplementing information about users or items (Zhang et al. 2024a; Ren et al. 2024; Xi et al. 2023). For example, RLMRec (Ren et al. 2024) develops a user/item profiling paradigm empowered by LLMs, and aligns the semantic space of LLMs with the representation space of collaborative relational signals through a cross-view alignment framework. LLMs are also employed to generate augmented training signals for coldstart items (Wang et al. 2024). Secondly, some works

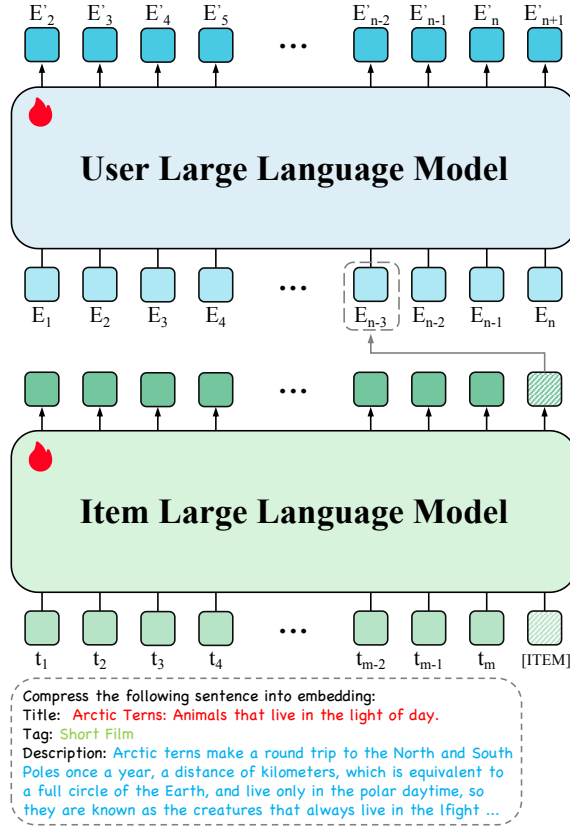


Figure 1: Architecture of Hierarchical Large Language Model. HLLM consists of two LLMs with non-shared parameters: Item LLM and User LLM. The Item LLM takes the text description of an item as input, appended with a special token `[ITEM]`, and outputs the item embedding. The User LLM inputs the item embeddings of the user’s historical interactions and predicts next item. All LLM parameters are trainable and optimized via next item prediction.

adapt the recommendation domain data into conversational formats (Bao et al. 2023; Friedman et al. 2023; Zhang et al. 2023; Yang et al. 2023; Zhai et al. 2023). Some approaches treat the recommendation task as a special form of instruction-following, inputting user historical behaviors in text form to the LLM to predict subsequent actions (Li et al. 2024). Lastly, there are also some works that have adapted LLMs for recommendation tasks, allowing their inputs or outputs to go beyond just textual forms. LLaRA (Liao et al. 2024) proposed a novel hybrid prompting method that integrates ID-based item embeddings with textual item features. LEARN (Jia et al. 2024) utilizes pre-trained LLMs to extract item features. LLMs are also adapted to multi-class classification or regression for rating prediction (Kang et al. 2023). However, these methods offer limited improvements compared to traditional recommendation models.

3 Method

In this section, we first introduce the problem formulation, and then propose Hierarchical Large Language Model

(HLLM) with a detailed explanation of how to adapt pre-trained large language models to recommendation systems, including item feature extraction and user interest modeling. Finally we discuss how to align HLLM with the objectives of recommendation systems, thereby significantly enhancing its performance on recommendation tasks.

Problem Formulation

We study the task of sequential recommendations, formulated as: Given a user $u \in \mathcal{U}$, a sequence of user u ’s historical interactions $U = \{I_1, I_2, \dots, I_n\}$ in chronological order, predict the next item I_{n+1} , where n is the length of U and $I \in \mathcal{I}$. Each item I has its corresponding ID and text information (e.g. title, tag, etc.), but the method proposed in this paper uses only the text information.

Hierarchical Large Language Model Architecture

Currently, a considerable number of LLM-based recommendation models flatten users’ historical behaviors into plain text inputs for the LLM (Kang et al. 2023; Yang et al. 2023; Li et al. 2024). This results in very long input sequences, and due to the self-attention module in LLMs, the complexity grows quadratically with the length of the input sequence. To reduce the burden of user sequence modeling, we adopt a hierarchical modeling approach called the Hierarchical Large Language Model (HLLM) that decouples item modeling from user modeling, as shown in Figure 1. Specifically, we first extract item features using the Item LLM, compressing the complex text descriptions into an embedding representation. Then, we model the user profile based on these item features with the User LLM. Additionally, to ensure better compatibility with pre-trained LLMs and to enhance scalability, we introduce minimal structural changes and design simple yet efficient training objectives. The following is a detailed introduction to item and user modeling.

Item LLM is proposed to extract item features. It takes as input the text description of an item and outputs an embedding representation. LLMs have demonstrated excellent performance in text comprehension, but their use has mostly been limited to text generation scenarios, with few works using them as feature extractors. Inspired by previous works (Devlin 2018; Neelakantan et al. 2022), a special token `[ITEM]` is added at the end of the item’s text description to extract features.

Specifically, as shown in Figure 1, for Item I we first flatten its corresponding textual attributes into the sentence T , and prepend it with a fixed prompt. After passing through the LLM tokenizer, we additionally append a special token `[ITEM]` at the end, thus the input token sequence for the Item LLM can be formulated as $\{t_1, t_2, \dots, t_m, [\text{ITEM}]\}$ where m represents the length of text tokens. The hidden state from the last layer corresponding to the special token `[ITEM]` is considered as the item embedding.

User LLM is designed to model user interests which is another key aspect of recommendation systems. The original user history sequence $U = \{I_1, I_2, \dots, I_n\}$ can be transformed into a historical feature sequence $\{E_1, E_2, \dots, E_n\}$

through the Item LLM, where E_i represents the item embedding of I_i . The User LLM takes this historical feature sequence as input and predict next item embedding based on a sequence of previous interactions. As shown in Figure 1, the output of the User LLM corresponding to E_i is E'_{i+1} , which is expected to be the embedding of I_{i+1} .

Unlike traditional LLMs with text-in and text-out formats, here both the input and output of the User LLM are item embeddings. Therefore, we discard the word embeddings from the pre-trained LLM but retain all other pre-trained weights. Experiments show that these pre-trained weights are very helpful for reasoning user interests.

Training for Recommendation Objectives

Existing LLMs are all pre-trained using general natural language corpora. Although they possess a wealth of world knowledge and strong reasoning abilities, there remains a considerable gap between their capabilities and those required by recommendation systems. Following the best practices of other works (Zhou et al. 2024; Touvron et al. 2023), we adopt supervised fine-tuning on top of the pre-trained LLM.

Recommendation systems can be divided into two categories, generative and discriminative recommendation. It is noteworthy that the proposed HLLM architecture is applicable to both types, requiring only appropriate adjustments to the training objectives. The following sections provide a detailed introduction to the training objectives for both categories.

Generative Recommendation Recent work (Zhai et al. 2024) has provided a successful generative recommendation solution, including both retrieval and ranking. Our approach differs from it in two major ways: the model architecture is upgraded to large language models with pre-trained weights, and the input features are changed from IDs to text-input LLM features. The above differences have minimal impact on the training and serving strategies, therefore, we largely follow approaches proposed in (Zhai et al. 2024).

For the training objective of generative recommendation, next item prediction is adopted, which aims to generate the embedding of the next item given the embeddings of the previous items in the user’s history. Specifically, the InfoNCE loss (Oord, Li, and Vinyals 2018) is used during training. For any prediction E'_i in the output sequence of the User LLM, the positive sample is E_i , and the negative samples are randomly sampled from the dataset excluding the current user sequence. The loss function can be formulated as:

$$\mathcal{L}_{gen} = - \sum_{j=1}^b \sum_{i=2}^n \log \frac{e^{s(E'_{j,i}, E_{j,i})}}{e^{s(E'_{j,i}, E_{j,i})} + \sum_k^N e^{s(E'_{j,i}, E_{j,i,k})}} \quad (1)$$

where s is the similarity function with a learnable temperature parameter, $E_{j,i}$ denotes the i -th item embedding produced by the Item LLM in the j -th user’s history interaction and $E'_{j,i}$ denotes the i -th item embedding predicted by the User LLM for the j -th user. N is the number of negative samples, $E_{j,i,k}$ represents the k -th negative embedding of

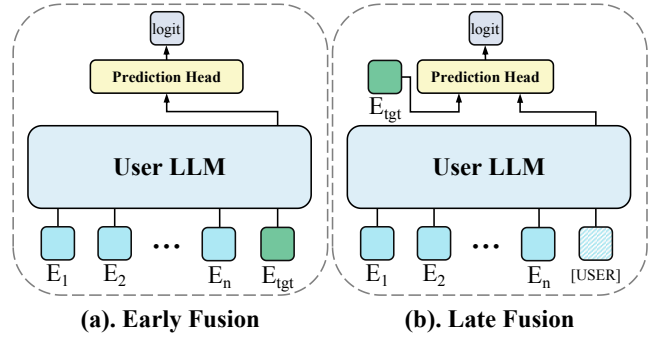


Figure 2: Two User LLM variants for discriminative recommendations.

$E'_{j,i}$. b represents the total number of users within the batch, n is the length of user history interactions.

Discriminative Recommendation Since discriminative recommendation models still dominate in the industry, we also present an application scheme for HLLM under discriminative recommendation models. The optimization objective of discriminative models is to judge, given a user sequence U and a target item I_{tgt} , whether the user is interested in the target item (e.g., by clicking, liking, purchasing, etc.).

As shown in Figure 2, there are two User LLM variants for discriminative recommendation, while keeping the Item LLM unchanged. **Early fusion** appends the target item embedding E_{tgt} to the end of the user’s historical sequence, then produces a high-order cross feature through User LLM, and finally inputs this cross feature into the prediction head to generate the final logits. **Late fusion**, on the other hand, first uses the User LLM to extract user features, which are independent of the target item, in a manner similar to the Item LLM feature extraction. A special token `[USER]` is added to the end of the user sequence to extract user representation. The user embedding and the target item embedding are then input together into the prediction head to predict the final logits. Early fusion, due to its deep integration of user interests and the target item, tends to perform better but is challenging to apply simultaneously across numerous candidates; conversely, late fusion is more efficient since different candidates share the same user features, but typically sees a performance decline.

The training objective of discriminative recommendation is usually a classification task, such as predicting whether a user will click, etc. For the binary classification example, the training loss is as follows:

$$\mathcal{L}_{cls} = -(y \cdot \log(x) + (1 - y) \cdot \log(1 - x)) \quad (2)$$

where y denotes the label of the training sample and x denotes the predicted logit.

Empirically, next item prediction can also be used as an auxiliary loss in discriminative models to further enhance performance. Hence, the final loss can be formulated as follows:

$$\mathcal{L}_{dis} = \lambda \mathcal{L}_{gen} + \mathcal{L}_{cls} \quad (3)$$

where λ controls the weight of the auxiliary loss.

Dataset	#User	#Item	#Interaction
Pixel200K	200,000	96,282	3,965,656
Pixel1M	1,001,822	100,541	19,886,579
Pixel8M	8,886,078	407,082	158,488,652
Books	694,898	686,624	10,053,086

Table 1: Statics of PixelRec and Amazon Book Reviews.

Item LLM	User LLM	R@5	R@10	N@5	N@10
Scratch	Scratch	3.330	5.063	2.199	2.755
Scratch	Pre-trained	3.556	5.416	2.371	2.969
Pre-trained	Scratch	3.521	5.331	2.358	2.940
Pre-trained	Pre-trained	3.755	5.581	2.513	3.100

Table 2: Ablation studies of pre-training on Pixel200K with HLLM-1B.

4 Experiments

In this section, we first introduce the basic experimental settings, and then numerous experiments are conducted to address the following research questions:

RQ1: Does the general pre-training of the LLM and the fine-tuning with recommendation objectives improve the final recommendation performance?

RQ2: Does HLLM have good scalability?

RQ3: Are the advantages of HLLM significant compared with other state-of-the-art models?

RQ4: How does the training and serving efficiency compare with ID-based models?

Finally, we demonstrate how to deploy HLLM in online scenarios and achieve real-world benefits.

Datasets and Evaluation Setup

For offline experiments, we evaluate HLLM on two large-scale datasets: PixelRec (including three subsets: 200K, 1M, and 8M) (Cheng et al. 2024), and Amazon Book Reviews (Books) (McAuley et al. 2015). Consistent with previous works (Cheng et al. 2024; Zhai et al. 2024), we adopt the same data preprocessing and evaluation protocols to ensure a fair comparison. A more detailed analysis of these datasets after preprocessing is presented in Table 1 and Figure 5. We utilize a leave-one-out approach to split the data into training, validation, and testing sets. Performance is measured using the metrics Recall@K (R@K) and NDCG@K (N@K). All open-source datasets are employed solely for training and evaluating in offline experiments.

Baselines and Training

For baselines, we use two ID-based sequential recommenders SASRec (Kang and McAuley 2018), and HSTU (Zhai et al. 2024). They are all aimed at industrial applications and boast state-of-the-art performance.

For offline experiments, the generative recommendation is used to stay consistent with other methods. For the online A/B test, discriminative recommendation is used to better

#Tokens	R@5	R@10	N@5	N@10	CSR $\uparrow$
0T	3.330	5.047	2.199	2.755	-
0.1T	3.539	5.142	2.399	2.915	46.11
1T	3.613	5.409	2.414	2.993	50.22
1T+chat	3.610	5.387	2.411	2.984	51.36
2T	3.650	5.510	2.466	3.063	51.64
3T	3.755	5.581	2.513	3.100	52.99

Table 3: The impact of different pre-training token counts on Pixel200K with HLLM-1B. “+chat” means SFT on conversation data. The CSR metric is the average performance on the common sense reasoning tasks.

Item LLM	User LLM	R@5	R@10	N@5	N@10
Frozen	Learnable	0.588	0.945	0.372	0.486
Learnable	Frozen	1.619	2.470	1.070	1.343
Learnable	Learnable	3.755	5.581	2.513	3.100
SASRec-1B		1.973	2.868	1.352	1.640

Table 4: Ablation studies of fine-tuning on Pixel200K with HLLM-1B.

align with the online system¹.

In HLLM-1B, we use TinyLlama-1.1B (Zhang et al. 2024b) for both Item LLM and User LLM. Correspondingly, in HLLM-7B, we utilize Baichuan2-7B (Baichuan 2023) for both. Due to resource constraints, HLLMs are trained only 5 epochs on PixelRec and Amazon Reviews while other models are trained 50 and 200 epochs, respectively. The learning rate is set to 1e-4. Each item’s text length is truncated to a maximum of 256. On PixelRec, following PixelNet (Cheng et al. 2024), we utilize a batch size of 512. The maximum sequence length is set to 10, and the ratio of positive to negative samples is 1:5632. On Books, we utilize a batch size of 128, set a maximum sequence length of 50, and the number of negative samples is 512.

For a fair comparison, we also implemented SASRec-1B (replacing its network structure with TinyLlama-1.1B) and HSTU-1B, which uses the same hidden size and number of layers as TinyLlama-1.1B but has only 462M parameters due to the elimination of the traditional FFN.

Pre-training and Fine-tuning (RQ1)

As clearly seen from Table 2, pre-trained weights are beneficial for HLLM, including both item feature extraction and user interest modeling. Furthermore, as shown in Table 3, the performance is positively correlated with the number of pre-trained tokens, indicating that the quality of pre-trained weights also impacts the recommendation task. However, supervised fine-tuning (SFT) on conversation data can result in slight negative effects, probably because world knowledge is primarily acquired during the pre-training stage, and SFT mainly enhances instruction-following abilities, which do not aid in recommendation tasks (Zhou et al. 2024).

¹Experiments demonstrated that most conclusions drawn from the academic dataset still hold true on large-scale industrial benchmarks.

Item Model	#Params	R@5	R@10	N@5	N@10
BERT-Base	110M	2.576	4.020	1.694	2.158
BERT-Large	340M	3.032	4.635	1.993	2.508
TinyLlama	1.1B	3.484	5.239	2.319	2.883

Table 5: Experiments with different sizes of the item model on Pixel200K. SASRec is used as the user model for all.

User Model	#Params	R@5	R@10	N@5	N@10
SASRec	4M	3.484	5.239	2.319	2.883
Llama-2L	0.1B	3.494	5.233	2.338	2.898
TinyLlama	1.1B	3.521	5.331	2.358	2.940

Table 6: Experiments with different sizes of the user model on Pixel200K. Llama-2L maintains the same architecture as Llama but uses only 2 decoder layers. TinyLlama-1.1B is used as the item model for all. All user models are trained from scratch.

It is also evident that fine-tuning both the Item LLM and User LLM is crucial for outperforming ID-based models, as shown in Table 4. When we freeze the Item LLM and only fine-tune the User LLM, using mean pooling of all token outputs in the last layer of TinyLlama-1.1B as item features, we find that the performance is very poor. This indicates that LLMs trained on predicting the next token are not directly suitable as feature extractors. Similarly, when we use an Item LLM that has been fine-tuned on Pixel200K and freeze the pre-trained User LLM, the performance remains critically low.

Scaling Up (RQ2)

The experimental results for increasing the model’s parameter count are shown in Table 5 and Table 6. It can be observed that the growth in the number of parameters for both Item LLM and User LLM consistently leads to performance improvements. Finally, we scale up both the Item LLM and User LLM from 1 billion parameters to 7 billion parameters on the Amazon Books. As shown in Table 7, this leads to further performance improvements, demonstrating that HLLM has excellent scalability.

To explore scalability of data volume, we sampled multiple different scales of data from Pixel8M for training, ranging from 0.1M to 8M in size. From Figure 3, it is evident that HLLM demonstrates remarkable scalability across various data volumes. With increasing data, significant enhancements in performance are observed, and no performance bottlenecks are observed at the current data scale.

We also conducted more comprehensive ablation experiments related to scaling up on a large-scale industrial recommendation dataset to demonstrate the scalability of the HLLM architecture, with detailed experimental results presented in the appendix.

HLLM vs. SOTA Methods (RQ3)

In Table 7, we compare the performance of HLLM with the current state-of-the-art models, including ID-based models such as SASRec (Kang and McAuley 2018) and

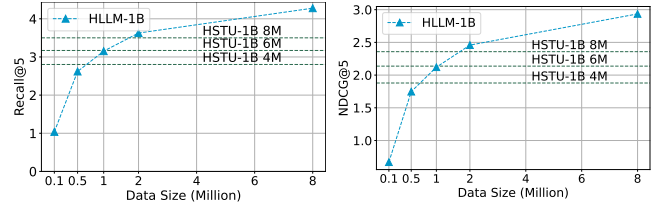


Figure 3: Experiments of HLLM’s performance at various data scales. Recall@5 and NDCG@5 are reported.

HSTU (Zhai et al. 2024), as well as the text-based model LEARN (Jia et al. 2024) on the Pixel8M and Amazon Book Reviews datasets. They all exhibit excellent performance and are dedicated to industrial practice.

It’s clear that HLLM holds a significant performance advantage, decisively outperforming other models on all metrics across all datasets. Under the same experimental settings, compared to the lowest-performing baseline, HLLM-1B shows an average improvement of **22.93%** on Pixel8M, and an even more significant average improvement of **108.68%** on Books. In contrast, ID-based models only show a maximum improvement of 5.37% on Pixel8M and 64.96% on Books.

Furthermore, it is notable that when ID-based models increase the number of negative samples and batch size, the performance improvements are relatively modest, especially in R@200 where HSTU-large only increases by **0.76**, while HLLM-1B increases by **2.44** under the same setting. By further increasing the model’s parameters, HLLM-7B achieves a significant improvement of **169.58%** compared to the baseline, which is highly impressive.

The table also shows that even with fully converged ID-based models, the gains from increasing parameters are minimal. On Pixel8M, both SASRec-1B and HSTU-1B show relatively modest improvements compared to smaller sizes, while on Books, SASRec-1B even experiences a decline in all metrics. In contrast, for HLLM, scaling up from HLLM-1B to HLLM-7B still results in corresponding performance improvements on recommendation tasks, demonstrating the superiority of the HLLM architecture.

Training and Serving Efficiency (RQ4)

Firstly, HLLM shows better training data efficiency than ID-based models. As shown in Figure 3, HLLM requires only one-sixth to one-fourth of the data volume to achieve performance on par with ID-based methods.

Previous extensive experiments have shown that fully fine-tuning the entire HLLM significantly improves performance but requires real-time encoding of all items during inference, which is inefficient. Thanks to the decoupling of item and user encoding in HLLM, our architecture can reduce computational complexity by caching item embeddings in advance. To demonstrate the feasibility of item caching, we pre-trained HLLM on sequences longer than 10 from the Pixel8M dataset, truncating sequences at the tenth position to avoid data leakage, covering 3 million users. Based on this pre-trained HLLM, we freeze the Item LLM and fine-tune only the User LLM on Pixel8M. Results in Ta-

Dataset	Method	R@10	R@50	R@200	N@10	N@50	N@200	Impv. (avg)
Pixel8M	SASRec _{vit} (2024)	3.589	-	-	1.941	-	-	-27.72%
	HSTU* (2024)	4.848	10.315	18.327	2.752	3.939	5.135	+0.0%
	SASRec*	5.083	10.667	18.754	2.911	4.123	5.331	+3.82%
	HSTU-1B*	5.120	11.010	19.393	2.879	4.159	5.411	+5.37%
	SASRec-1B*	5.142	10.899	19.044	2.915	4.166	5.383	+4.83%
	HLLM-1B (Ours)	6.129	12.475	21.179	3.539	4.919	6.221	+22.93%
Amazon Books	SASRec (2018)	3.06	7.54	14.31	1.64	2.60	3.62	+0.0%
	LEARN (2024)	4.07	9.79	18.74	2.24	3.71	4.83	+34.42%
	HSTU-large (2024)	4.78	10.82	19.08	2.62	3.93	5.17	+47.80%
	SASRec*	5.35	11.91	21.02	2.98	4.40	5.76	+64.96%
	SASRec-1B*	5.09	11.11	19.45	2.86	4.17	5.42	+55.68%
	HSTU-large*	5.00	11.29	20.13	2.78	4.14	5.47	+55.61%
	HSTU-1B*	5.25	12.03	21.60	2.89	4.36	5.80	+64.37%
	HLLM-1B (Ours)	6.97	14.61	24.78	3.98	5.64	7.16	+108.68%
	HSTU-large [†] *	6.49	12.22	19.81	3.99	5.24	6.38	+88.94%
	HLLM-1B-Scratch [†] (Ours)	6.85	13.95	23.19	4.02	5.56	6.95	+103.65%
	HLLM-1B [†] (Ours)	9.28	17.34	27.22	5.65	7.41	8.89	+166.42%
	HLLM-7B [†] (Ours)	9.39	17.65	27.59	5.69	7.50	8.99	+169.58%

Table 7: Performance comparison of HLLM with SOTA models. SASRec_{vit} means SASRec uses the ViT as an image encoder for item encoding and trained by BCE loss from (Cheng et al. 2024). * indicates the result is reproduced by us. † indicates the number of negative samples and the batch size are increased from 512 and 128 to 28k and 512, respectively. “Scratch” indicates both Item LLM and User LLM are trained from scratch.

Method	R@5	R@10	N@5	N@10
HSTU-1B	3.501	5.120	2.358	2.879
HLLM-1B _{cache}	3.585	5.218	2.432	2.958
HLLM-1B	4.278	6.106	2.935	3.524

Table 8: Experiments on the effectiveness of item caching. HLLM-1B_{cache} utilizes a pre-trained item HLLM to extract item features, but the parameters are frozen.

ble 8 show that while freezing the Item LLM leads to some metric decreases, performance still exceeds ID-based models, proving item caching is more effective. Given that user behaviors in industrial scenarios far exceed the number of items, HLLM’s training and serving costs can match those of ID-based models. Notably, our pre-training data constitutes less than half of Pixel8M, with some items not appearing in the pre-training data, yet we still achieve respectable performance. Experiments on industrial data show that as the amount of pre-training data increases, the gap between the item caching and the full fine-tuning is largely narrowed.

Online A/B Test

Apart from offline experiments, HLLM is also targeted at and successfully applied in real-world industrial practices. For simplicity, flexibility, and to better align with the online system, we adopted HLLM-1B, using the discriminative recommendation approach with the late fusion variant for optimization. Considering the balance between performance and efficiency, our training process is divided into the following three stages:

Stage I: End-to-end training of all HLLM parameters, including Item LLM and User LLM with discriminative loss. The user history sequence length is truncated to 150 to ac-

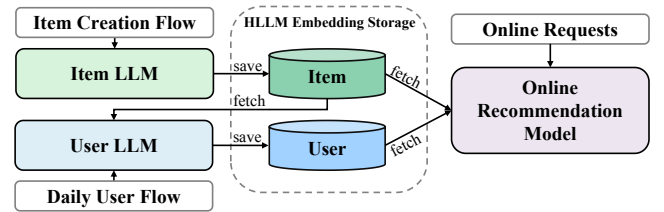


Figure 4: An overview of the online system.

celerate training.

Stage II: We first use the Item LLM trained in Stage I to encode and store the embeddings of all items in the recommendation system. We then continue to train only the User LLM by retrieving the necessary item embeddings from storage. Since this stage only trains the User LLM, it significantly reduces the training demand, allowing us to extend the user sequence length from 150 to 1000, further enhancing the effectiveness of the User LLM.

Stage III: After extensive data training in the first two stages, the HLLM model parameters are no longer updated. We extract features for all users which are then combined with item LLM embeddings and other existing features and fed into the online recommendation model for training.

Regarding serving, as shown in Figure 4, item embeddings are extracted when they are created, and user embeddings are updated on a daily basis only for users who had activity the previous day. Embeddings of items and users are stored for online model training and serving. Under this approach, the inference time of the online recommendation system is virtually unchanged.

Finally, we test HLLM in online A/B experiments of the ranking task. Key metrics have shown a significant increase of **0.705%**.

5 Conclusion

In this paper, we propose a novel Hierarchical Large Language Model (HLLM) architecture designed to enhance sequential recommendations. HLLM leverages LLMs to extract item features and model user interests, effectively integrating pre-training knowledge into the recommendation system, and it is proved that fine-tuning with recommendation objectives is essential. HLLM exhibited excellent scalability with larger model parameters. Experiments demonstrated that HLLM outperforms traditional ID-based models, achieving state-of-the-art results on academic datasets. Real-world online A/B testing further validated HLLM's practical efficiency and applicability, marking a significant advancement in the field of recommendation systems.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Alteschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Baichuan. 2023. Baichuan 2: Open Large-scale Language Models. *arXiv preprint arXiv:2309.10305*.
- Baltescu, P.; Chen, H.; Pancha, N.; Zhai, A.; Leskovec, J.; and Rosenberg, C. 2022. Itemsage: Learning product embeddings for shopping recommendations at pinterest. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2703–2711.
- Bao, K.; Zhang, J.; Zhang, Y.; Wang, W.; Feng, F.; and He, X. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*, 1007–1014.
- Cheng, Y.; Pan, Y.; Zhang, J.; Ni, Y.; Sun, A.; and Yuan, F. 2024. An Image Dataset for Benchmarking Recommender Systems with Raw Pixels. In *Proceedings of the 2024 SIAM International Conference on Data Mining (SDM)*, 418–426. SIAM.
- Cui, Z.; Ma, J.; Zhou, C.; Zhou, J.; and Yang, H. 2022. M6-rec: Generative pretrained language models are open-ended recommender systems. *arXiv preprint arXiv:2205.08084*.
- Devlin, J. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- Friedman, L.; Ahuja, S.; Allen, D.; Tan, Z.; Sidahmed, H.; Long, C.; Xie, J.; Schubiner, G.; Patel, A.; Lara, H.; et al. 2023. Leveraging large language models in conversational recommender systems. *arXiv preprint arXiv:2305.07961*.
- Goldberg, D.; Nichols, D.; Oki, B. M.; and Terry, D. 1992. Using collaborative filtering to weave an information tapestry. *Communications of the ACM*, 35(12): 61–70.
- Guo, H.; Tang, R.; Ye, Y.; Li, Z.; and He, X. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. *arXiv preprint arXiv:1703.04247*.
- Hidasi, B.; Karatzoglou, A.; Baltrunas, L.; and Tikk, D. 2015. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939*.
- Jia, J.; Wang, Y.; Li, Y.; Chen, H.; Bai, X.; Liu, Z.; Liang, J.; Chen, Q.; Li, H.; Jiang, P.; et al. 2024. Knowledge Adaptation from Large Language Model to Recommendation for Practical Industrial Application. *arXiv preprint arXiv:2405.03988*.
- Kang, W.-C.; and McAuley, J. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*, 197–206. IEEE.
- Kang, W.-C.; Ni, J.; Mehta, N.; Sathiamoorthy, M.; Hong, L.; Chi, E.; and Cheng, D. Z. 2023. Do llms understand user preferences? evaluating llms on user rating prediction. *arXiv preprint arXiv:2305.06474*.
- Koren, Y.; Bell, R.; and Volinsky, C. 2009. Matrix factorization techniques for recommender systems. *Computer*, 42(8): 30–37.
- Li, J.; Wang, M.; Li, J.; Fu, J.; Shen, X.; Shang, J.; and McAuley, J. 2023a. Text is all you need: Learning language representations for sequential recommendation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 1258–1267.
- Li, L.; Zhang, Y.; Liu, D.; and Chen, L. 2023b. Large language models for generative recommendation: A survey and visionary discussions. *arXiv preprint arXiv:2309.01157*.
- Li, Y.; Zhai, X.; Alzantot, M.; Yu, K.; Vulić, I.; Korhonen, A.; and Hammad, M. 2024. CALRec: Contrastive Alignment of Generative LLMs For Sequential Recommendation. *arXiv preprint arXiv:2405.02429*.
- Liao, J.; Li, S.; Yang, Z.; Wu, J.; Yuan, Y.; Wang, X.; and He, X. 2024. Llara: Large language-recommendation assistant. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1785–1795.
- McAuley, J.; Targett, C.; Shi, Q.; and Van Den Hengel, A. 2015. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*, 43–52.
- Neelakantan, A.; Xu, T.; Puri, R.; Radford, A.; Han, J. M.; Tworek, J.; Yuan, Q.; Tezak, N.; Kim, J. W.; Hallacy, C.; et al. 2022. Text and code embeddings by contrastive pre-training. *arXiv preprint arXiv:2201.10005*.
- Ning, L.; Liu, L.; Wu, J.; Wu, N.; Berlowitz, D.; Prakash, S.; Green, B.; O'Banion, S.; and Xie, J. 2024. User-LLM: Efficient LLM Contextualization with User Embeddings. *arXiv preprint arXiv:2402.13598*.
- Oord, A. v. d.; Li, Y.; and Vinyals, O. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*.
- OpenAI. 2022. Introducing ChatGPT. <https://openai.com/blog/chatgpt>.
- Ren, X.; Wei, W.; Xia, L.; Su, L.; Cheng, S.; Wang, J.; Yin, D.; and Huang, C. 2024. Representation learning with large language models for recommendation. In *Proceedings of the ACM on Web Conference 2024*, 3464–3475.

- Sarwar, B.; Karypis, G.; Konstan, J.; and Riedl, J. 2001. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*, 285–295.
- Shin, K.; Kwak, H.; Kim, S. Y.; Ramström, M. N.; Jeong, J.; Ha, J.-W.; and Kim, K.-M. 2023. Scaling law for recommendation models: Towards general-purpose user representations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, 4596–4604.
- Sun, F.; Liu, J.; Wu, J.; Pei, C.; Lin, X.; Ou, W.; and Jiang, P. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*, 1441–1450.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Wang, J.; Lu, H.; Caverlee, J.; Chi, E. H.; and Chen, M. 2024. Large Language Models as Data Augmenters for Cold-Start Item Recommendation. In *Companion Proceedings of the ACM on Web Conference 2024*, 726–729.
- Wang, R.; Fu, B.; Fu, G.; and Wang, M. 2017. Deep & cross network for ad click predictions. In *Proceedings of the AD-KDD’17*, 1–7.
- Wang, R.; Shivanna, R.; Cheng, D.; Jain, S.; Lin, D.; Hong, L.; and Chi, E. 2021. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the web conference 2021*, 1785–1797.
- Wu, L.; Zheng, Z.; Qiu, Z.; Wang, H.; Gu, H.; Shen, T.; Qin, C.; Zhu, C.; Zhu, H.; Liu, Q.; et al. 2023. A survey on large language models for recommendation. *arXiv preprint arXiv:2305.19860*.
- Xi, Y.; Liu, W.; Lin, J.; Cai, X.; Zhu, H.; Zhu, J.; Chen, B.; Tang, R.; Zhang, W.; Zhang, R.; et al. 2023. Towards open-world recommendation with knowledge augmentation from large language models. *arXiv preprint arXiv:2306.10933*.
- Yang, F.; Chen, Z.; Jiang, Z.; Cho, E.; Huang, X.; and Lu, Y. 2023. Palr: Personalization aware llms for recommendation. *arXiv preprint arXiv:2305.07622*.
- Zhai, J.; Liao, L.; Liu, X.; Wang, Y.; Li, R.; Cao, X.; Gao, L.; Gong, Z.; Gu, F.; He, M.; et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152*.
- Zhai, J.; Zheng, X.; Wang, C.-D.; Li, H.; and Tian, Y. 2023. Knowledge prompt-tuning for sequential recommendation. In *Proceedings of the 31st ACM International Conference on Multimedia*, 6451–6461.
- Zhang, C.; Sun, Y.; Chen, J.; Lei, J.; Abdul-Mageed, M.; Wang, S.; Jin, R.; Park, S.; Yao, N.; and Long, B. 2024a. SPAR: Personalized Content-Based Recommendation via Long Engagement Attention. *arXiv preprint arXiv:2402.10555*.
- Zhang, J.; Xie, R.; Hou, Y.; Zhao, W. X.; Lin, L.; and Wen, J.-R. 2023. Recommendation as instruction following: A large language model empowered recommendation approach. *arXiv preprint arXiv:2305.07001*.
- Zhang, P.; Zeng, G.; Wang, T.; and Lu, W. 2024b. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*.
- Zhou, C.; Liu, P.; Xu, P.; Iyer, S.; Sun, J.; Mao, Y.; Ma, X.; Efrat, A.; Yu, P.; Yu, L.; et al. 2024. Lima: Less is more for alignment. *Advances in Neural Information Processing Systems*, 36.
- Zhou, G.; Mou, N.; Fan, Y.; Pi, Q.; Bian, W.; Zhou, C.; Zhu, X.; and Gai, K. 2019. Deep interest evolution network for click-through rate prediction. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, 5941–5948.
- Zhou, G.; Zhu, X.; Song, C.; Fan, Y.; Zhu, H.; Ma, X.; Yan, Y.; Jin, J.; Li, H.; and Gai, K. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 1059–1068.

Tag	Title	Description	Length	R@5	R@10
✓			64	0.082	0.196
	✓		64	3.520	5.317
✓	✓		64	3.610	5.439
✓	✓	✓	64	3.647	5.502
✓	✓	✓	256	3.755	5.581

Tag	Title	Description	Length	N@5	N@10
✓			64	0.049	0.086
	✓		64	2.348	2.926
✓	✓		64	2.415	3.003
✓	✓	✓	64	2.430	3.026
✓	✓	✓	256	2.513	3.099

Table 9: Ablation studies of text length and richness on Pixel200K.

Method	R@5	R@10	N@5	N@10
Mean Pooling	3.386	5.159	2.257	2.826
[ITEM] Token	3.484	5.239	2.319	2.883

Table 10: Ablation studies of Item LLM feature extraction method on Pixel200K. Mean pooling refers to using the mean pooling of hidden states from the last layer of the Item LLM as the item features. SASRec is used as the user model.

A More Experiments on Academic Datasets

Textual Input Length and Richness of Item LLM

By default, we input all types of text information with a length of 256. Here, we conduct ablation experiments on text length and richness. Table 9 shows that the text content has a significant impact on the final performance. Richer text content and longer text lengths allow the Item LLM to extract more detailed item features, better differentiate between items, and more effectively aid the User LLM in modeling user interests.

Method of Item LLM Feature Extraction

To enable LLMs trained on next token prediction to have feature extraction capabilities, we add a special token [ITEM] at the end of the text input. Another feasible feature extraction approach is to take the average of the hidden states from the final layer of the LLM to represent the features of the entire sentence. Table 10 shows the comparison results of these two methods. As can be seen, using the [ITEM] token is better than mean pooling.

Sequence Length of User LLM

We explore the impact of input sequence length of User LLM on HLLM’s recommendation performance in Table 11. Similar to other sequential recommenders, HLLM can also benefit from expanding the length of the input sequence. Although the table shows only modest performance gains with increasing sequence length, we suspect this is likely because user sequence lengths are generally quite short in the academic dataset as shown in Figure 5. As shown in Appendix B, in the real-world industrial systems, where

Length	R@5	R@10	R@50	R@200
10	5.201	7.564	16.220	28.776
30	5.235	7.605	16.293	28.837
50	5.238	7.631	16.416	28.959

Length	N@5	N@10	N@50	N@200
10	3.538	4.299	6.176	8.052
30	3.556	4.319	6.205	8.081
50	3.568	4.338	6.244	8.119

Table 11: Experiments on the sequence length of User LLM on Pixel1M.

Input Features	R@5	R@10	N@5	N@10
Item ID	4.105	6.082	2.773	3.409
LLM Emb	5.201	7.564	3.538	4.299
LLM Emb + Item ID	5.154	7.501	3.504	4.260
LLM Emb + Timestamp	5.779	8.319	3.953	4.770

Table 12: Ablation studies of input features of User LLM on Pixel1M. LLM Emb represents the item features extracted using the Item LLM based on textual descriptions.

user behavior sequences are typically very long, extending the sequence length allows HLLM to achieve stable performance improvement.

Compatibility with ID-based Features

In the previous sections, we primarily modeled item and user features based on the textual descriptions of items. Most current recommendation systems, however, still rely on ID features, including not only Item IDs but also features like actions, timestamps, and item categories in ID form. Here, we present a compatibility solution for integrating HLLM with ID features, and demonstrate that complementary ID features, when combined with item descriptions, can indeed bring significant improvements to HLLM, further highlighting its application value in industrial environments.

Here, we choose the raw item IDs and timestamps as ID features for validation. The item IDs are transformed into id embeddings through an embedding lookup table. The behavior’s timestamp is first split into specific year, month, day, hour, minute, and second components, obtaining the timestamp embedding as Algorithm 1. We perform sum pooling with the ID features and item LLM embeddings before inputting them into the User LLM. The prediction target during training remains the item embedding extracted by the Item LLM, and the experimental results are shown in Table 12. The introduction of item IDs actually results in a slight decrease in performance, likely because the item IDs do not provide incremental information beyond what is already captured by the textual descriptions, which comprehensively describe the item’s characteristics and are sufficiently extracted by the Item LLM. However, the improvement resulting from the introduction of timestamps is very pronounced, as timestamps complement the textual descriptions. This also demonstrates that our method can be compatible with ID-based features.

Sequence Length	AUC
200	0.7429
500	0.7446
1,000	0.7458

Table 13: Experiments on the sequence length of User LLM on the industrial dataset.

Item LLM	User LLM	AUC
1B	1B	0.7458
1B	7B	0.7498
7B	1B	0.7517
7B	7B	0.7533

Table 14: Experiments with different sizes of Item LLM and User LLM on the industrial dataset.

B Scaling Up of HLLM on Industrial Dataset

More extensive experiments are conducted on a large-scale industrial dataset to evaluate the scalability of HLLM.

Douyin has a vast number of users and recommendation candidates, with extensive records of user behavior. We construct a dataset comprising 30 million samples from the past 3 years’ logs. Each sample includes only the user’s historical click sequence, the target item, and a label indicating whether the item was clicked or not. We validate the effectiveness of HLLM in a discriminative recommendation system, using AUC as the evaluation metric, and verifying scalability from two aspects: the sequence length of User LLM, and the parameters of both Item LLM and User LLM.

Sequence Length of User LLM

The length of user behavior sequences in the industrial dataset is shown in Figure 5. And table 13 shows the impact of user sequence length, with HLLM’s performance steadily increasing as the sequence length grows. This illustrates HLLM’s substantial potential in modeling users with longer sequences.

Parameters of Item LLM and User LLM

Table 14 illustrates the impact of the parameters of HLLM in industrial scenario. For both Item LLM and User LLM, AUC consistently increases with the growth in the number of parameters.

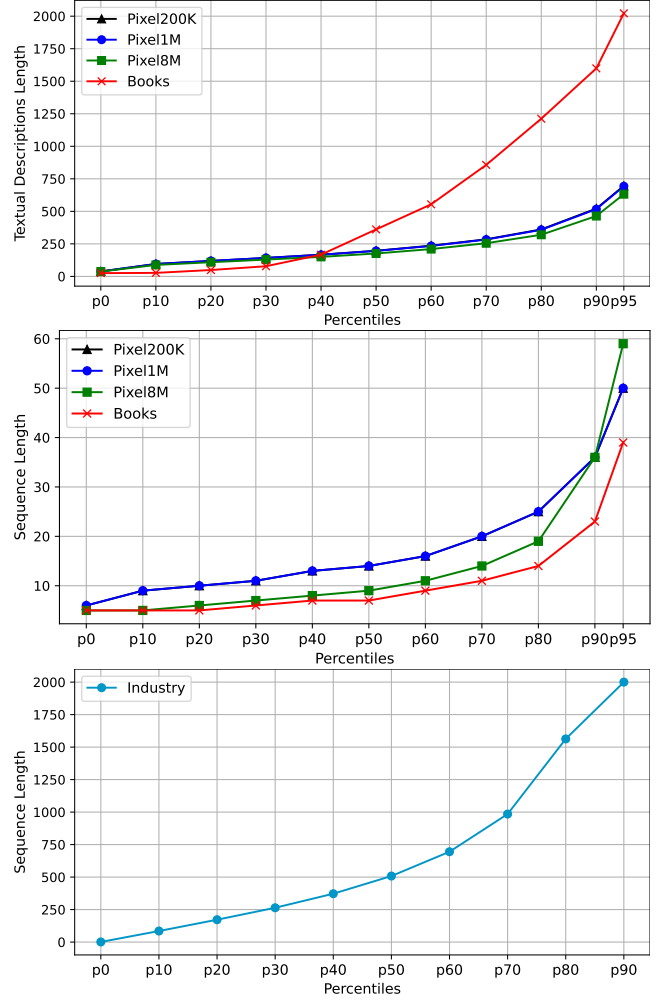


Figure 5: Distribution of textual descriptions (flattening all attributes) and sequence lengths in Pixel200K, Pixel1M, Pixel8M, Amazon Book Reviews and industrial scenario. Since Pixel200K is randomly sampled from Pixel1M, their distributions are consistent. We truncate the sequence length to a maximum of 2,000 for industrial data, hence p90 is exactly 2,000.

Algorithm 1: Pseudo code of timestamp processing in a PyTorch-like style.

```
1  class TSEmbedding(nn.Module):
2      def __init__(self, time_num=6, time_dim=512, user_dim=2048):
3          super().__init__()
4          # Control the precision of time, such as 4 to the hour, and 6 to the second.
5          self.time_num = time_num
6          self.time_embeddings = nn.ModuleList(nn.Embedding(x, time_dim) for x in [2100, 13, 32, 24, 60, 60])
7          # Projection from time_dim to user_dim
8          self.merge_time = MLP(time_dim * time_num, user_dim)
9
10     def split_time(self, timestamps: List) -> List:
11         # Split timestamps into specific components.
12         # (seq) -> (seq, 6)
13         split_time = []
14         for time in timestamps:
15             dt = datetime.datetime.fromtimestamp(time)
16             split_time.append([dt.year, dt.month, dt.day, dt.hour, dt.minute, dt.second])
17         return split_time
18
19     def forward(self, timestamps: List) -> torch.tensor:
20         # Times: timestamp of each item in List format (bs, seq)
21         # (bs, seq) -> (bs, seq, 6)
22         time_seq = torch.tensor([self.split_time(x) for x in timestamps])
23         # (bs, seq, 6) -> [(bs, seq, time_dim)] * time_num
24         time_emb = [self.time_embeddings[i](time_seq[... , i]) for i in range(self.time_num)]
25         # [(bs, seq, time_dim)] * time_num -> (bs, seq, time_dim * time_num)
26         time_emb = torch.cat(time_emb, dim=-1)
27         # (bs, seq, time_dim * time_num) -> (bs, seq, user_dim)
28         time_emb = self.merge_time(time_emb)
29         return time_emb
```
